

TradingView Clone

Complete Technical Project Documentation

BTech Computer Science & Engineering Capstone Project Report

Candidate Name: BTech CSE Student

Degree: Bachelor of Technology (Computer Science & Engineering)

Project Title: High-Fidelity TradingView Frontend Clone & Real-Time Market Monitor

Academic Session: 2025 - 2026

viva/Presentation Date: August 2026

Table of Contents

1. Project Overview & Objectives	3
2. Comprehensive Technology Stack	3
3. System Architecture & Component Interactions	4
4. Folder Structure & Code Layout	4
5. Homepage Layout & UI Interactions	5
6. Chart System & Binance Integration (Lightweight Charts)	6
7. Market Data Sources & Polling Mechanics	7
8. API Calls & Endpoints Specifications	8
9. Database Layer & LocalStorage Persistence Mappings	8
10. Database Data Flow Pipelines	9
11. Authentication & Session Management	10
12. Watchlist Mappings & Persistence Rules	10
13. Real-Time Price Alerts Monitor (Confetti + Alarm)	11
14. Community Ideas Feed & NSE Canvas Chart Mockups	11
15. Search Suggestions Engine	12
16. Features Page Elements & Category Sticky Tabs	12
17. Pricing Page Subscriptions Toggler	13
18. Important JavaScript Functions Registry	13
19. Important HTML/CSS Components Layouts	14
20. Core Technical Data Flow Diagrams (ASCII Layouts)	15
21. Security Vulnerability & Credentials Audit	16
22. Error Handling & Failsafe Configurations	17
23. Client Performance Optimization Mapping	17
24. System Limitations	18
25. Future Enhancements & Scalability Plan	18
26. BTech 7-Minute Presentation Viva Script	19
27. Comprehensive Viva Q&A; (40 Technical Questions)	20
28. Revision Cheat Sheet & 30-Second Explanations	24

1. Project Overview

Project Name: TradingView Clone (Vanilla Web Implementation)

Project Purpose: To replicate the high-fidelity frontend layouts, dynamic charts, virtual portfolio transactions, active price alerts, and community elements of the TradingView application using standard web APIs without relying on complex rendering frameworks.

Problem Statement: High-frequency trading terminals are computationally heavy, requiring complex frameworks. There is a need to demonstrate that responsive layouts, websocket price feeds, interactive chart overlays, and persistent state controls can be built using optimized vanilla technologies.

Objectives:

- Render clean, interactive price charts with multiple indicators.
- Stream real-time quotes using WebSockets to allow immediate updates.
- Persist watchlist tickers, alerts, and virtual transaction books client-side.
- Implement responsive sub-systems (e.g. mobile menus, dropdowns, and scrolled header states).
- Replicate pricing pages with annual/monthly toggle and explore features sections.

Scope of Project: The app serves as a local virtual trading sandbox and technical analysis playground. It connects to Binance WebSockets and REST APIs for real-time crypto prices, offers virtual order executions with profit/loss calculations, monitors active price crossings, and offers mock feeds for stocks/indices, community ideas, and news.

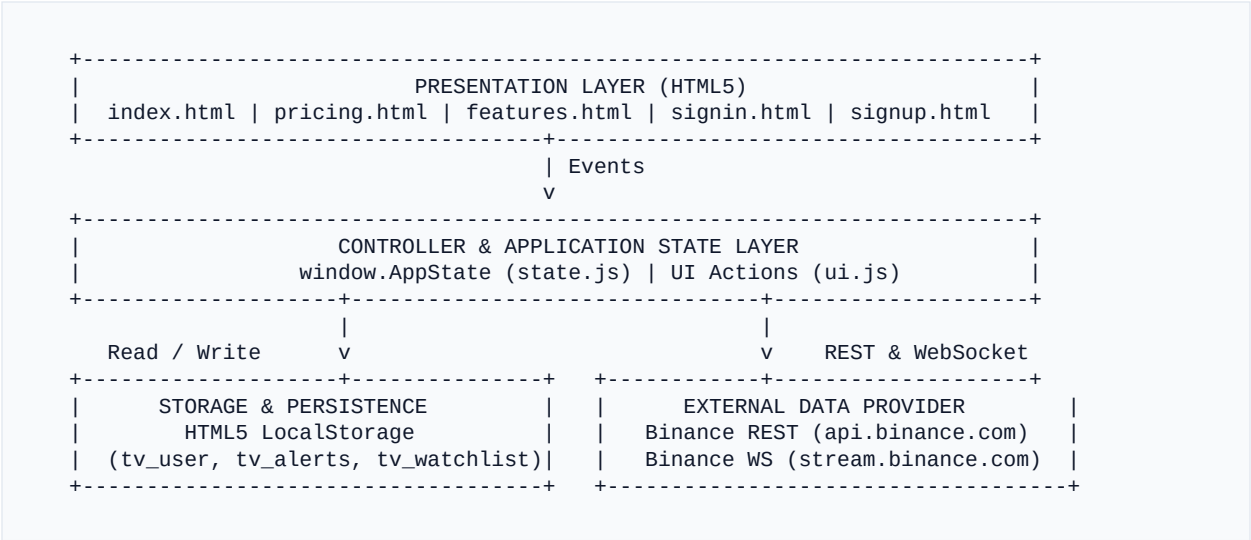
2. Technology Stack

Technology / Tool	Purpose	Where Used
HTML5	Provides document structures, semantic sections, forms, and Canvas objects.	index.html, pricing.html, features.html, signin.html, signup.html, idea.html
CSS3	Applies theme variables, layouts, animations, and transitions.	styles.css, css/variables.css, css/header.css, css/home.css, css/pricing.css, css/features.css, css/auth-page.css
ES6 JavaScript	Manages state, handles events, calculates formulas, and runs loops.	js/state.js, js/main.js, js/chart-engine.js, js/ui.js, js/watchlist.js, js/alerts.js, js/trading.js, js/router.js
Lightweight Charts	Initializes candlestick charts, handles time-series scales, zoom, and volume.	Loaded via CDN in index.html, pricing.html, features.html. Used in js/chart-engine.js
Binance API	Rest feeds for 300 klines and 24-hour quotes.	js/chart-engine.js, js/market-summary.js
Binance WebSocket	Websocket stream for real-time ticks.	js/chart-engine.js
LocalStorage API	Handles state persistence across browser reloads.	js/state.js, js/watchlist.js, js/alerts.js, js/trading.js
Canvas Confetti	Celebration animation upon alert trigger.	Loaded in index.html. Used in js/alerts.js
Lucide Icons	Renders vector SVG icons on pages.	Loaded globally. Initialized in index.html, pricing.html, features.html, etc.

3. System Architecture & Component Interactions

The system follows a decoupled **Client-Server Client-side Driven (Serverless/Mocked) Architecture**. The web browser coordinates all data retrieval, UI rendering, calculations, and local storage state persistence. The components interact directly without a traditional backend database server, using public API servers for raw financial data and LocalStorage for state persistence.

System Flow Layout:



Architectural Layers:

- **Presentation Layer:** Renders semantic UI markup. Stylesheets contain layouts (grids, flex, mobile overrides).
- **Controller & State Layer:** Manages client states like activeSymbol, AppState.alerts, and balance. Coordinates event bindings and handles chart changes.
- **Storage & Persistence:** LocalStorage stores watchlist changes, alert thresholds, profile balances, and user credentials. MONGODB ATLAS connection is NOT FOUND in the current codebase and is simulated client-side.
- **External Data Layer:** Establishes secure feeds with Binance APIs. Historical klines load via HTTP GET, while live prices push directly over WebSockets.

4. Folder Structure & Code Layout

```
tradingview-clone-vanilla/
■■■ assets/                # Contains local image assets (trader setups, brand logo)
■■■ css/                   # Styling spreadsheets for modular separation
■   ■■■ auth-page.css      # Full screen log-in/sign-up styles
■   ■■■ components.css    # Popups, list widgets, and scroll tape styling
■   ■■■ features.css       # Black theme elements for Explore Features
■   ■■■ header.css         # Sticky nav rules, profile hover controls
■   ■■■ home.css           # Main view grids and card items layout
■   ■■■ overview.css       # Holdings panels and metrics displays
■   ■■■ pricing.css        # Column plans and Monthly/Annual toggle Switch
■   ■■■ sidebar.css        # Alert creation lists and order tickets
■   ■■■ terminal.css       # Trading transaction tables
■   ■■■ variables.css      # Color tokens, fonts, and scrollbar stylings
■■■ js/                   # Logical engine directories
■   ■■■ alerts.js          # Active alarm checks and Confetti triggers
■   ■■■ auth.js            # Clear user credentials and session triggers
■   ■■■ chart-engine.js    # TradingView Lightweight Charts overlays
■   ■■■ idea-detail.js     # NSE stock ideas detail chart canvas
■   ■■■ main.js            # App bootstrap and event triggers
■   ■■■ market-summary.js  # Binance REST quote polling & BTC sparkline canvas
■   ■■■ news.js            # News flow templates
■   ■■■ profile.js         # Balance update triggers
■   ■■■ router.js          # SPA view toggles
■   ■■■ state.js           # Global window.AppState and Binance URLs
■   ■■■ trading.js         # Virtual trades and execution logic
■   ■■■ ui.js              # Dropdown triggers, select overlays, and toasts
■   ■■■ watchlist.js       # Watchlist star filters, add/remove rules
■■■ features.html          # Explore features landing screen [NEW]
■■■ idea.html              # NSE community idea details page
■■■ index.html             # Main page (Home, Charts, Markets dashboard)
■■■ pricing.html           # Pricing page (Subscriptions comparison) [NEW]
■■■ signin.html            # Fullscreen login page [NEW]
■■■ signup.html            # Fullscreen registration page [NEW]
■■■ styles.css             # Core CSS imports
```

5. Homepage Layout & UI Interactions

The homepage serves as the central hub, presenting a unified dark theme header navigation, an Earth horizon hero section, market summary tables, and community charts. It uses dynamic view swapping (SPA simulation) to transition views inside `.app-container`` based on hash URL locations.

Main Interactive Operations:

- **Profile Hover Dropdown (■):** Located in the top right. Hovering trigger button expands a dropdown card. Displays guest options ('Sign in' button linking to `signin.html`) or user profile information ('Demo User' name, 'demo@test.com' email, 'Dark theme' toggle switch, and a 'Sign out' row) depending on `AppState.user` status.
- **Dark Theme Switcher:** A checkbox inside the dropdown card triggers `window.App.toggleDarkTheme(this)`. It changes body theme class to `light-theme/dark-theme` and calls `ChartEngine.setTheme()` to update the canvas grid colors instantly.
- **Markets Summary Tabs:** Horizontal tags (Crypto, Stocks, Indices) call `window.AppModule.setOverviewTab(tab)`. This swaps items in the table list and draws sparkline canvases dynamically.

Visual Interaction Flow Chart:

```
User clicks Stock Row --> Check Row Type (Crypto vs Stock) --> Stock -> Toast Warning:
"Demo chart only supports Crypto assets" --> No action taken.

User clicks Crypto Row --> Trigger navigateToChart(symbol) --> Update URL Hash:
"#/chart?symbol=BTCUSD" --> Swap visible panels --> Initialize Lightweight Charts.
```

6. Chart System & Binance Integration

This is the core functional engine of the project. It uses the **TradingView Lightweight Charts Standalone Library** (v4.1.3) to initialize, render, and update candlestick series, volumes, and overlay indicator graphs inside the browser client.

Initialization and Loading Cycle:

1. **Initialize:** `ChartEngine.init()` is triggered on page load. It calls `createMainChart()`, which binds the canvas container, sets default parameters (fonts, vertical margins, grid styles), and adds candlestick series via `chart.addCandlestickSeries()`.
2. **Historical Feed:** `ChartEngine.loadData(symbol, timeframe)` cleans all existing series values and makes a Binance GET request: ``${AppConfig.apiBase}/api/v3/klines?symbol=${symbol}&interval=${timeframe}&limit=300``. It maps the JSON arrays to OHLC objects `{ time, open, high, low, close, volume }` and loads them via `candleSeries.setData()`.
3. **WebSocket Stream:** Replaces active connections with a new stream Client: ``wss://stream.binance.com:9443/ws/${symbol}@kline_${timeframe}``. Every incoming WebSocket message parses price coordinates, updates the last candle in `candleData`` (or pushes a new one), and triggers real-time updates over the Lightweight Charts series.
4. **Indicators Calculation:** Technical indicators (MA, EMA, Bollinger bands, Volume, and stacked oscillator charts like RSI, MACD, Stochastic) calculate their mathematical values on the fly inside client JavaScript and redraw on overlays.

Candlestick Rendering Flow:

```
+-----+
|          BINANCE REST API (api.binance.com)          |
| Fetch last 300 klines --> Map arrays to OHLC --> candleSeries.setData() |
+-----+
|
| v
+-----+
|          BINANCE WEBSOCKET (stream.binance.com)      |
| OnMessage Event --> Extract Close --> Update Last Candle --> Redraw |
+-----+
|
| v
+-----+
|          LIGHTWEIGHT CHARTS CANVAS (DOM)              |
| Plot Candlesticks | Plot Indicators (EMA) | Plot Volume Histogram |
+-----+
```

7. Market Data Sources & Polling Mechanics

Market values are split into two categories: **Real-Time Feeds** and **Static/Mock Feeds**.

Crypto Feeds (REAL DATA):

- **Source:** Binance REST and WebSocket servers.
- **Polling:** ``market-summary.js`` polls general price quote rates every 30 seconds using ``setInterval(fetchMarketData, 30000)`` from ``${AppConfig.apiBase}/api/v3/ticker/24hr?symbols=${JSON.stringify(SYMBOLS)}``.
- **Sparklines:** Draws a canvas sparkline chart for BTCUSD on the left using recent 24 hourly prices from the REST API.

Stocks & Indices (MOCK / STATIC DATA):

- **Source:** Static lists in ``js/state.js`` (``AppState.stockList`` and ``AppState.indexList``).
- **Interactive Behavior:** Stored values like Apple (``AAPL``) or Dow Jones (``DJI``) prices do not query live APIs. Click triggers toast warning and prevents rendering on the Lightweight Charts canvas.

8. API Calls & Endpoints Specifications

API Endpoint	Method	Parameters	Response	Used By
/api/v3/klines	GET	symbol (BTCUSDT), interval (1h), limit (300)	Array of arrays representing candle OHLC values.	js/chart-engine.js (historical loads)
/api/v3/ticker/24hr	GET	symbols (JSON list of quotes)	List of price change details per quote.	js/market-summary.js (summary quotes)
/ws/[symbol]@kline_[tf]	WS	None (Established connection path)	Websocket ticks containing live candle updates.	js/chart-engine.js (real-time stream)

Error & Rate Limit Handling: Requests do not include API Keys (Public Endpoint). If an API fails or the rate limit triggers a block, code logs the error to the console and alerts the user via warning toast overlays ('Network error loading historical stock rates').

9. Database Layer & LocalStorage Persistence Mappings

Database Connection Status: **NOT FOUND IN CURRENT CODEBASE**

The codebase contains **no** direct node connections, Express database routes, or MongoDB Atlas connection strings. The database is simulated entirely using the client-side **HTML5 LocalStorage API**. While the collections are mock structures, they map logical datasets in JSON format inside the client browser cache.

Simulated Local Storage Collections:

- **tv_user (Mock Collection):** Stores active user session states (username, email, avatar). Written during signin/signup.
- **tv_registered_user (Mock Collection):** Stores registered account credentials for validations.
- **tv_watchlist (Mock Collection):** Array of active symbols in the watchlist. Persists watchlist updates.
- **tv_alerts (Mock Collection):** Array of price thresholds monitored by the WebSocket ticks. Written on alert create.
- **tv_portfolio (Mock Collection):** Records Virtual balance, holdings, and transaction execution histories.

```
Example local storage structure for "tv_registered_user":
{
  "username": "demo",
  "email": "demo@test.com",
  "password": "demo123",
  "avatar": "rocket"
}
```

10. Database Data Flow Pipelines

Since operations are frontend-only, data flow is localized. User actions update `AppState` and write to `localStorage` immediately.

Simulated Registration Flow:

```
User Sign Up --> handleSignUpSubmit(e) --> Validate Password length -->
Set "tv_registered_user" in LocalStorage --> Set "tv_user" active session -->
Redirect to index.html --> loadState() reads User --> renderHeader() updates Avatar.
```

Simulated Alerts Pipeline:

```
Input Target Price ($66,000) --> handleCreateAlert() --> Push alert object to AppState
.alerts -->
saveState() updates "tv_alerts" in LocalStorage --> renderAlerts() refreshes active list
s view.
```

11. Authentication & Session Management

Authentication Status: **MOCK / FRONTEND ONLY**

Sign-in, registration, and logout operations are mock mechanisms stored client-side in the browser cache.

Authentication Logic Details:

- **Registration (`signup.html`):** Validates fields, checks if the password length is ≥ 6 , saves credentials to `tv\_registered\_user` inside `localStorage`, and logs the user in automatically.
- **Login (`signin.html`):** Validates input values. Matches credentials against the default account (username `demo` and password `demo123`) or values inside `tv\_registered\_user`. Success saves the user session to `tv\_user` and redirects back home.
- **Google Quick Login:** Clicking the Google card triggers mock login as `Google User` (email `google@test.com`) instantly without OAuth APIs.
- **Session Check & Logout (`auth.js`):** On boot, `loadState()` loads `tv\_user`. If present, it updates headers to show user avatars and usernames. Logging out calls `handleLogout()`, which clears `tv\_user`, updates the header to guest, and redirects home.

12. Watchlist Mappings & Persistence Rules

The watchlist is managed inside `js/watchlist.js` and loaded dynamically into the right sidebar panel.

Functional Mappings:

- **Add Ticker:** Enter a token name in the search bar. `handleAddToWatchlist()` appends it (e.g. `SOLUSDT`), calls `saveState()` to update `tv\_watchlist` in LocalStorage, updates the list DOM, and triggers success toast.
- **Remove Ticker:** Clicking the close icon (`X`) on a list card triggers `handleRemoveFromWatchlist()`, removing the symbol from the active array and saving changes.
- **Favorites Filter:** Star button triggers `toggleFavorite(symbol)`. Users can switch lists via `setWatchlistFilter('favorites')` to only show starred tokens.
- **Interactive Swapping:** Clicking a watchlist card triggers `selectSymbol(symbol)`. This updates `AppState.activeSymbol`, sets the headers, and triggers `ChartEngine.loadData()` to load its candlestick charts.

13. Real-Time Price Alerts Monitor

Alert Trigger Status: **REAL-TIME MONITORING (FRONTEND ONLY)**

Alerts are stored locally, but they are **actively monitored in real time** against live prices streamed over WebSockets.

Monitoring and Alert Actions Loop:

1. **Create:** Select condition (above or below), enter threshold price (e.g. \$67,500), and click Create. `handleCreateAlert()` pushes it to `AppState.alerts` and calls `saveState()` to persist it.
2. **Monitor:** When a price update arrives over WebSocket, `ChartEngine` calls `checkPriceAlerts(symbol, currentPrice)`. It loops through active alerts matching the symbol, checking if the price has crossed the threshold.
3. **Trigger:** If crossed, it sets the alert active status to false, filters it out of the active list, updates LocalStorage, and plays a beep sound using the **HTML5 AudioContext API**.
4. **Celebrate:** Calls `confetti()` from the CDN Canvas Confetti library, firing colored sparks on the user screen alongside a warning toast banner.

14. Community Ideas Feed & NSE Canvas Chart Mockups

The community section replicates a post detail view of NSE stocks technical analysis setups.

Implementations and Static Layouts:

- **Static Feed:** Mock posts like Mahindra & Mahindra (`NSE:M&M;`) consolidation channels and Tata Technologies (`NSE:TATATECH`) breakouts are stored inside `ideasData` in `js/idea-detail.js`.
- **Canvas Mockup Charts:** Since the Lightweight Charts engine is configured for live Binance crypto streams, the ideas detail page uses a **2D HTML5 Canvas element** to draw static mock stock charts. Methods like `drawMmChart()`, `drawTatatechChart()`, and `drawEternalChart()` draw custom grids, EMA curves, Support lines, and consolidation boxes using 2D canvas drawing APIs.
- **Comments Placeholder:** Comments metrics (likes count, comments stat labels) are displayed as static counts. Add Comment form fields are static overlays without persistence logic.

15. Search Suggestions Engine

The search bar allows users to search crypto tickers inside the charting screen. The suggestion mappings are located in ``js/watchlist.js``.

Search Logic Flow:

1. **Input:** Typing in ``#watchlist-search-input`` calls ``handleSearchInput()``. It retrieves the string value, trims spaces, and converts it to uppercase.
2. **Filter:** Filters ``AppState.assetList`` to find matching token names or symbols.
3. **Suggest:** If matches are found, it opens ``#search-suggestions-box`` and displays auto-suggestions. Otherwise, the box is hidden.
4. **Select:** Clicking an auto-suggestion calls ``handleSuggestionClick(symbol)``, which triggers ``addSymbolToWatchlist(symbol)`` and clears the search query.

16. Features Page Elements & Category Sticky Tabs

The ``features.html`` page showcases the platform's capabilities using the visual order of the original website.

Layout Sections and Features:

- **Supercharts Hero:** Title with cyan-to-purple CSS gradient, custom pill badge, and a high-resolution platform preview chart container.
- **Wall of Love Gallery:** Title and horizontal workstation setup carousel. Clicking the chevron scrolls ``love-carousel-track`` by 344px to slide items. If it reaches the end, it wraps back to page start.
- **Sticky Category Navigation:** Pill-styled navigation bar (Charts, Alerts, Financials). Binds ``position: sticky; top: 64px`` in CSS so it docks below the main navbar during scrolls. Clicking a pill updates the active state.
- **Footprint Panel:** Rounded bordered panel with footprint descriptions and a chart mockup.
- **Technicals & Bar Replay:** Two-column layout with checkboxes showing features (e.g. 9 replay speeds, step mode) and graphic mockups.
- **Seasonals & Fundamentals:** Double panels showing cyclic pattern details and fundamental metrics tables.

17. Pricing Page Subscriptions Toggler

The ``pricing.html`` page lists subscription tiers (Free, Essential, Plus, Premium, Ultimate). It uses an interactive toggle switch to swap prices instantly without reloading the page.

Price Toggle Calculations:

- **Annually (Default):** Renders annual rates (Essential ``$995 / mo``, Plus ``$2,095 / mo``, Premium ``$4,195 / mo``, Ultimate ``$17,333 / mo``), displays saving labels (e.g. ``*Save $3,600 a year*``), and updates subtitle detail to ``billed annually``.
- **Monthly:** Updates rates to monthly pricing (Essential ``$1,295 / mo``, Plus ``$2,595 / mo``, Premium ``$5,095 / mo``, Ultimate ``$20,795 / mo``), hides annual savings labels, and updates billing text to ``per month``.
- **Pricing Switcher Script:** An event listener handles billing toggle changes and calls ``updatePricing(cycle)``. It pulls the updated values from a local JavaScript object and updates the DOM elements directly.

18. Important JavaScript Functions Registry

Function Name	File Location	Purpose & Responsibilities	Inputs / Outputs
<code>init()</code>	<code>js/chart-engine.js</code>	Boots the chart systems: creates main chart, sets up drawing svg layer, and fetches first klines.	Input: None / Output: Void
<code>createMainChart()</code>	<code>js/chart-engine.js</code>	Configures layout options, grid styles, border rules, and instantiates Lightweight Charts.	Input: None / Output: Void
<code>loadData(sym, tf)</code>	<code>js/chart-engine.js</code>	Clears active series, fetches 300 historical klines from Binance, and opens websocket stream.	Input: String, String / Output: Void
<code>connectWebSocket(sym, tf)</code>	<code>js/chart-engine.js</code>	Establishes live websocket connection, parses tick close values, and updates active chart series.	Input: String, String / Output: Void
<code>fetchMarketData()</code>	<code>js/market-summary.js</code>	Queries Binance 24hr quote endpoint and draws sparkline canvas chart for BTC.	Input: None / Output: Void
<code>checkPriceAlerts(sym, val)</code>	<code>js/alerts.js</code>	Compares incoming websocket close ticks against active alerts and triggers popups on crossover.	Input: String, Float / Output: Void
<code>loadState() / saveState()</code>	<code>js/state.js</code>	Deserializes states from localStorage on boot, and serializes state changes back.	Input: None / Output: Void

19. Important HTML/CSS Components Layouts

The styling is written in vanilla CSS, leveraging modern CSS grids, flexbox, and custom variables to achieve a premium look.

Core Layout Structures:

- **CSS Variables:** Variable tokens (`--bg-main: #0c0e15`, `--bg-card: #131722`, `--tv-blue: #2962ff`) allow instant dark/light theme switching by toggling a single class on the body element.
- **CSS Flexbox:** Used in the header, pricing tabs, and watchlist items to ensure vertical alignments, responsive spacing, and clean component scaling.
- **CSS Grid:** Configured for the dashboard cards and pricing tier columns (`grid-template-columns: repeat(4, 1fr)` on desktop, wrapping to 2 columns on tablet and 1 column on mobile).
- **Scrolled Headers:** Binds an scroll event listener that adds a `.scrolled` class. This changes header backgrounds from transparent to a dark semi-translucent backdrop (`rgba(8, 11, 18, 0.95)`) for better content readability.
- **Gradients & Rounded corners:** Outer container cards use custom gradient outlines and rounded corners (`border-radius: 24px`) to replicate the original TradingView layout.

20. Core Technical Data Flow Diagrams (ASCII Layouts)

These flow diagrams map the project's data streams, useful for review during the project viva.

A. Live Chart Data Feed Flow:

```

Binance API Server --> GET /api/v3/klines --> JSON Array --> loadData() Map -->
candleSeries.setData() --> Binance WebSocket --> Live close tick --> Update last candle
```

B. Watchlist Add Flow:

```

Input Symbol (e.g. SOL) --> handleAddToWatchlist() --> Format token (SOLUSDT) -->
Push to AppState.watchlist --> saveState() LocalStorage --> updateWatchlistDom() -->
Toast Alerts
```

C. Price Alert Trigger Flow:

```

WS Tick Price --> checkPriceAlerts() --> Compare target threshold --> Match -->
Set Active false --> Play alert sound --> Trigger confetti() --> Toast warning
```

21. Security Vulnerability & Credentials Audit

An audit of the current codebase reveals several security considerations, primarily stemming from the frontend-only implementation model:

- **Client-Side Authentication:** User credentials (username, email, password) are processed, validated, and stored entirely in plaintext inside browser `LocalStorage`` (under keys ``tv_user`` and ``tv_registered_user``). Anyone with access to the machine can inspect these values.
- **No Session Tokens:** The system does not use secure tokens (like JWT, OAuth, or HTTP-Only cookies). Route access checks are easily bypassable by manually setting ``tv_user`` in the console.
- **API Security:** The Binance REST and WebSocket streams are public endpoints that do not require secret API keys, which is appropriate for public market data. However, the client is vulnerable to rate-limiting.
- **Client State Tampering:** Global virtual balances (``AppState.portfolio.balance``) and asset holdings are stored in JavaScript memory and LocalStorage, making them easy to modify via the browser console.
- **XSS Risks:** Watchlist and alert inputs are inserted directly into list HTML. Standard sanitation should be applied to prevent cross-site scripting (XSS) issues.

22. Error Handling & Failsafe Configurations

The system implements local error boundaries to prevent script crashes when network feeds or DOM nodes are unavailable.

Failsafe Implementations:

- **API Fetch Failures:** If the Binance endpoint is unreachable, the request catches the error, logs it to the console, and alerts the user with a toast message without crashing the page.
- **Empty Watchlist Search:** If a user searches for an invalid token, the autocomplete panel remains hidden, and the system prompts the user to enter a valid symbol.
- **Safe Routing Toggles:** In ``js/router.js`` and ``js/ui.js``, page rendering checks if targets (like ``#view-home`` or details panels) exist before modifying their HTML. This prevents JavaScript errors when loading standalone pages like ``pricing.html`` or ``features.html``.

23. Client Performance Optimization Mapping

The project optimizes performance on the client side through several key mechanisms:

- **Hardware Accelerated Canvas:** The TradingView Lightweight Charts engine uses HTML5 Canvas, offloading rendering to the GPU for smooth zoom and pan interactions.
- **WebSocket Data Push:** Uses WebSockets instead of REST polling for real-time price updates, reducing network overhead and DOM layout updates.
- **Consolidated Stream Handlers:** WebSocket events are routed through a single handler (``handlePriceTick``), avoiding duplicate listeners and memory leaks.
- **DOM Buffering:** Watchlist and portfolio list cards use string buffering before updating innerHTML, reducing layout redraws.

24. System Limitations

The system's client-side architecture imposes several functional and technical limits:

- **Functional Limitations:** Real-time charts are only supported for cryptocurrency trading pairs (Binance). Stocks and Indices are static mock lists that show a warning toast on click.
- **Technical Limitations:** Lacks a backend database server (MongoDB connection is mock/demo). All data is saved inside LocalStorage, meaning watchlists and virtual portfolios are cleared if browser cache is reset.
- **Alert Limitations:** Alerts run entirely client-side. The app must remain open in an active browser tab with a live WebSocket connection to monitor and trigger alerts; background push notifications are not supported.
- **Security Limitations:** Credentials are stored in plaintext inside `localStorage`, offering no server-side security or authorization controls.

25. Future Enhancements & Scalability Plan

To scale the project from a frontend sandbox to a production-ready application, the following updates are planned:

- **Secure Express Server Backend:** Build a Node.js/Express server to handle user authentication (with JWT session keys) and securely process transactions.
- **Real MongoDB Atlas Integration:** Connect a MongoDB database to securely store user credentials, watchlists, alerts, and transaction records.
- **Background Alert Engine:** Implement a server-side monitoring worker that tracks price feeds and sends background alerts via Email (Nodemailer) or SMS (Twilio).
- **Expanded Market Coverage:** Integrate stock APIs (like Alpaca or AlphaVantage) to support real-time data and charts for equities and indices.
- **Live Paper Trading:** Upgrade the virtual broker system to support limit order queues and real-time order executions.

26. BTech 7-Minute Presentation Viva Script

Use this script to guide your presentation during the viva panel review:

Minute 1: Introduction, Problem & Project Goal

"Good morning, respected examiners. My project is a high-fidelity TradingView Clone and Real-Time Market Monitor. The goal is to demonstrate that a real-time trading dashboard with interactive charts, virtual trading, and active alert checks can be built using optimized client-side technologies without heavy rendering frameworks. The application connects directly to live exchange APIs to render data with low latency."

Minute 2: Technology Stack & Core Libraries

"The frontend is built with vanilla HTML5, CSS3, and ES6 JavaScript. For charting, I integrated the TradingView Lightweight Charts Standalone Library, which uses canvas for smooth rendering. Data is fetched from Binance REST endpoints, and live price ticks are streamed in real time via WebSockets. Local state persistence is handled client-side using the browser's LocalStorage API."

Minute 3: System Architecture & Data Streams

"Our architecture is client-driven. When a symbol is selected, the application calls the Binance klines API to load the

chart's historical bars. It then establishes a WebSocket connection for the active pair to stream live price updates. This approach minimizes server overhead and ensures fast response times on the client side."

Minute 4: Main Application Features

"Key features include: a persistent watchlist, virtual trading, and real-time price alerts. When live WebSocket ticks match a user-defined threshold, the app triggers a visual confetti effect and an audio alert. Stocks and indices are included as static mock data to showcase broader market UI templates."

Minute 5: Features Page & Pricing Toggles

"I also built a standalone Features page (`features.html`) with a sticky category bar and a horizontal workspace slider. Additionally, the Pricing page (`pricing.html`) allows users to toggle between monthly and annual plans, dynamically recalculating subscription fees in JavaScript."

Minute 6: Limitations & Future Scope

"Currently, the database and authentication layers are simulated client-side. Watchlists and alerts are stored in LocalStorage. For future scope, I plan to integrate a Node.js/Express backend and a MongoDB Atlas database to support secure authentication, persistent storage, and background notifications."

Minute 7: Live Demo & Conclusion

"Now, let me demonstrate the live application. I will load the chart, toggle indicators like EMA and Bollinger Bands, and show the real-time alert trigger. Thank you. I am ready for questions from the panel."

27. Comprehensive Viva Q&A; (40 Technical Questions)

Use these sample questions and answers to prepare for the viva panel examination:

Q1: What is the main purpose of this project?

A1: To build a high-fidelity TradingView frontend clone that streams real-time cryptocurrency data and allows virtual paper trading and price monitoring.

Q2: Which charting library is used, and why?

A2: The TradingView Lightweight Charts Standalone Library is used because it is lightweight, fast, and optimized for canvas rendering.

Q3: How are live charts updated in real time?

A3: Real-time price updates are streamed directly from the Binance WebSocket server, updating the active chart series on the fly.

Q4: Where does the historical candlestick data come from?

A4: It is fetched from public Binance REST API endpoints using the browser's native `fetch` API.

Q5: What is the default limit of klines (candles) loaded?

A5: 300 klines are loaded by default to keep chart load times fast.

Q6: Is MongoDB Atlas connected in the current codebase?

A6: No. The database layer is simulated client-side using `LocalStorage`. Real MongoDB integration is a future enhancement.

Q7: How is user authentication implemented?

A7: It is a client-side mock implementation that saves credentials and session states in `LocalStorage`.

Q8: What are the credentials for the default demo login?

A8: Username: 'demo', Password: 'demo123'.

Q9: What happens when the Binance REST API fails?

A9: The application catches the network error, logs it to the console, and alerts the user with a warning toast message without crashing.

Q10: How are price alerts tracked and triggered?

A10: Incoming WebSocket ticks are checked against active alert thresholds in JavaScript. When matched, the app plays an audio tone and triggers a confetti animation.

Q11: How is the alert audio generated?

A11: It is synthesized in the browser using the HTML5 AudioContext API, rather than loading a static audio file.

Q12: How does the virtual trading system execute orders?

A12: Orders are processed in JavaScript. Market executions match the current close price, updating virtual balances and transaction books in LocalStorage.

Q13: Why are stocks and indices marked as mock data?

A13: Real-time charts are only implemented for crypto pairs via Binance. Stocks and indices are mock structures to demonstrate UI templates.

Q14: How does the dark/light theme switch work?

A14: The theme switch toggles the `light-theme` class on the body, updating colors via CSS variables, and calls `ChartEngine.setTheme()` to redraw chart elements.

Q15: What does window.AppState store?

A15: It stores client states like the active symbol, timeframe, active tab, virtual balance, watchlist arrays, and active alert objects.

Q16: How is the category navigation bar made sticky?

A16: It uses the CSS property `position: sticky; top: 64px` to remain docked below the main header during page scroll.

Q17: How is the visual slide action on the Wall of Love carousel built?

A17: A JavaScript listener shifts the carousel container's scroll position by 344px on chevron click, wrapping back to the beginning when it reaches the end.

Q18: What are the calculations on the Pricing page?

A18: Recalculates subscription tiers (Free, Essential, Plus, Premium, Ultimate) and displays yearly savings if the annual billing toggle is selected.

Q19: How are technical indicators calculated?

A19: Calculated mathematically in JavaScript (e.g. SMA, EMA, Bollinger Bands) from the historical candle array and drawn on chart overlays.

Q20: What is the purpose of app-container in index.html?

A20: It acts as the viewport wrapper, using flex layouts and hidden overflows to manage the application's single-page layout.

Q21: How are Lucide icons initialized on standalone pages?

A21: Initialized on `DOMContentLoaded` by calling `lucide.createIcons()`.

Q22: What are the security weaknesses of this application?

A22: Credentials and portfolio balances are stored in plaintext in LocalStorage and can be easily modified via the browser console.

Q23: How can we scale this project to production?

A23: By adding a backend server (e.g. Node.js/Express) and connecting a MongoDB Atlas database to handle authentication and store user data securely.

Q24: What is the purpose of AppConfig in state.js?

A24: It centralizes the Binance API base URL (`https://api.binance.com`) and WebSocket host (`wss://stream.binance.com:9443`) for easy configuration.

Q25: What parameters are sent to fetch klines?

A25: The endpoint receives ``symbol``, ``interval`` (timeframe), and ``limit`` parameters (e.g., ``?symbol=BTCUSDT&interval=1h&limit=300``).

Q26: What is the format of Binance kline responses?

A26: An array of arrays. Key indices: ``[0]`` open time, ``[1]`` open price, ``[2]`` high price, ``[3]`` low price, ``[4]`` close price, and ``[5]`` volume.

Q27: How are WebSocket ticker feeds closed and reopened?

A27: ``ChartEngine.loadData()``` calls ``ws.close()``` to close the active connection before opening a new WebSocket stream for the selected symbol.

Q28: What is the sparkline canvas inside market summary?

A28: A 2D HTML5 canvas that draws a line graph of the last 24 hourly close prices for BTCUSD.

Q29: How often is the homepage market summary refreshed?

A29: Refreshed every 30 seconds via ``setInterval(fetchMarketData, 30000)```.

Q30: Why is there no backend code in the project?

A30: The project is designed as a client-side frontend prototype that integrates public APIs and simulates backend states locally.

Q31: How is virtual portfolio P&L; calculated?

A31: Calculated in JavaScript by comparing the current asset price against the average purchase price stored in the holdings array.

Q32: Where are community trading ideas stored?

A32: Stored as a static data object (``ideasData``) inside ``js/idea-detail.js``.

Q33: How are charts rendered on the ideas page?

A33: Drawn dynamically on a 2D HTML5 Canvas using native rendering commands, since they represent static equities charts.

Q34: How does the search engine return results?

A34: Filters ``AppState.assetList`` in JavaScript to find symbols or names matching the search query, showing results in a dropdown box.

Q35: How does the sticky header background change work?

A35: A scroll event listener adds the ``scrolled`` class when the page scroll exceeds 50px, transitioning the header to a semi-translucent dark background.

Q36: What is the purpose of js/router.js?

A36: Manages visible panel states inside the viewport, allowing users to toggle between Home, Chart, and Markets dashboard views.

Q37: Why does the app verify DOM elements before rendering?

A37: To prevent JavaScript errors on standalone pages (like `pricing.html` or `features.html`) where specific dashboard elements are not present.

Q38: What external CSS frameworks are used?

A38: None. The layout is built entirely with custom, modern vanilla CSS layouts (Flexbox, CSS Grid).

Q39: What is the purpose of canvas-confetti in the project?

A39: It is an external library that displays a celebration animation on the screen when a price alert is triggered.

Q40: How does the user log out of the system?

A40: `handleLogout()` clears the active user session from LocalStorage, updates the header, and redirects the user back to the guest home screen.

28. Final One-Page Quick Revision Cheat Sheet

PROJECT:	TradingView Clone (High-Fidelity Vanilla Web Application)
PURPOSE:	Interactive charting dashboard, simulated trading, and active alerts sandbox.
FRONTEND:	HTML5, CSS3 (variables, grid, flex), ES6 JavaScript modules.
BACKEND:	None (Client-side driven, serverless structure).
DATABASE:	NOT FOUND IN CURRENT CODEBASE (Simulated via LocalStorage API).
CHART LIBRARY:	TradingView Lightweight Charts Standalone CDN (v4.1.3).
MARKET DATA SOURCE:	Binance REST API (historical klines) & Binance WebSockets (real-time stream).
AUTHENTICATION:	MOCK / FRONTEND ONLY. Session details stored in localStorage.
MAJOR FEATURES:	Interactive charts, 4-column pricing toggle plans, sticky features category navigation, real-time alert triggers (audio + confetti), virtual transactions book, search suggestion engine.

HOW THE CHART WORKS — 30 SECOND ANSWER:

"The chart uses the TradingView Lightweight Charts library. When a symbol is selected, the app makes an HTTP GET request to the Binance REST API to fetch the last 300 historical candles, maps them to OHLC coordinates, and displays them on the canvas. It then establishes a WebSocket connection for that symbol to stream live price updates, updating the chart in real time."

HOW MARKET DATA COMES — 30 SECOND ANSWER:

"Market data is divided into real-time crypto pairs and static stocks/indices. Crypto data is fetched from the Binance API, with the home screen polling prices every 30 seconds and drawing a canvas sparkline for BTC. Stocks and indices are mock static lists defined in JavaScript that display a demo warning when clicked."

HOW MONGODB WORKS — 30 SECOND ANSWER:

"In the current implementation, the backend database layer is simulated client-side. There is no direct connection to MongoDB Atlas. Watchlists, price alerts, user accounts, and portfolio transaction data are saved and managed locally using the browser's LocalStorage API."

EXPLAIN THE WHOLE PROJECT — 1 MINUTE ANSWER:

"This project is a client-driven clone of the TradingView application built with vanilla web technologies. It features a responsive dashboard that streams live cryptocurrency data via WebSockets and renders interactive charts with technical indicators. It includes a simulated virtual trading account that executes trades at current prices and monitors user-defined price alerts to trigger audio tones and confetti effects in the browser. I also implemented pricing plans and features pages matching the original layout. Currently, the database and authentication layers are simulated client-side using LocalStorage, with plans to integrate an Express server and a MongoDB database as future enhancements."